

LAPORAN TUGAS PRAKTIKUM 2

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Krisna Aji Pratama

FAKULTAS TEKNIK

PRODI TEKNIK INFORMATIKA

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi sistem operasi modern menuntut pemahaman mendalam mengenai konsep manajemen proses dan komunikasi antar proses (Inter-Process Communication/IPC) sebagai fondasi fundamental dalam arsitektur komputasi terkini. Implementasi multi-processing dan mekanisme sinkronisasi menjadi kritikal dalam pengembangan aplikasi konkuren yang efisien, terutama dalam lingkungan sistem terdistribusi dan real-time computing. Praktikum ini dirancang untuk memberikan pengalaman empiris dalam mengimplementasikan sistem manajemen proses canggih dan aplikasi komunikasi real-time menggunakan primitive sistem operasi Linux.

1.2 Rumusan Masalah

1. Implementasi sistem manajemen proses dengan multiple child processes yang menjalankan tugas berbeda secara paralel
2. Koordinasi dan komunikasi antara parent process dengan child processes menggunakan mekanisme IPC
3. Pembangunan aplikasi chat real-time menggunakan shared memory dengan sinkronisasi yang aman

1.3 Tujuan

1. Mengimplementasikan sistem manajemen proses dengan fitur fork() dan pipe communication untuk koordinasi tugas
2. Membangun aplikasi chat menggunakan shared memory IPC dengan semaphore synchronization
3. Mengevaluasi reliability dan performance kedua sistem dalam lingkungan Linux aktual

BAB II

PEMBAHASAN

2.1 Sistem management proses

2.1.1 Membuat Sistem Management Proses

Program C yang diimplementasikan merepresentasikan simulasi sistem manajemen proses yang sophisticated dengan arsitektur multi-process menggunakan system call `fork()`. Implementasi ini menciptakan tiga child processes terdistink yang dieksekusi secara konkuren, masing-masing mengimplementasikan algoritma komputasi yang berbeda: bubble sort untuk pengurutan array, linear search untuk pencarian elemen, dan mathematical calculation untuk enumerasi bilangan genap. Mekanisme koordinasi yang canggih diwujudkan melalui kombinasi pipe communication untuk inter-process communication (IPC) dan `waitpid` system call untuk proses synchronisation. Parent process berfungsi sebagai controller yang mengorkestrasi eksekusi, mengumpulkan metadata eksekusi melalui pipe channels, dan melakukan comprehensive reporting yang mencakup PID identification, precise execution time measurement dalam milidetik menggunakan `gettimeofday()`, serta termination status analysis melalui macro `WIFEXITED` dan `WTERMSIG`. Setiap child process menjalankan independent computational tasks pada shared memory array dengan dimensi 10.000 elemen, dimana hasil komputasi dikomunikasikan secara terstruktur melalui dedicated pipe connections, memastikan reliable data transfer dan process isolation sambil maintaining system stability melalui proper resource cleanup dan error handling mechanisms.

- Membuat file `management_process`



```
asus@LAPTOP-BV04OHP1: ~/tugas_os$ cd tugas_os
asus@LAPTOP-BV04OHP1: ~/tugas_os$ touch management_process.c
asus@LAPTOP-BV04OHP1: ~/tugas_os$ nano management_process.c
```

- Compile dan jalankan

```

asus@LAPTOP-BV04OHP1: ~/tugas_os
asus@LAPTOP-BV04OHP1:~/tugas_os$ touch management_process.c
asus@LAPTOP-BV04OHP1:~/tugas_os$ nano management_process.c
asus@LAPTOP-BV04OHP1:~/tugas_os$ gcc -o management_process management_process.c
asus@LAPTOP-BV04OHP1:~/tugas_os$ ./management_process
Parent Process PID: 2557
Membuat 3 child processes...

Parent Process mengkoordinasi child processes...
=====
Child Process 1:
PID: 2558
Task: Sorting
Execution Time: 87 ms
Status: Completed successfully
Result: Array terurut (contoh elemen pertama: 0)
=====
Child Process 2:
PID: 2559
Task: Searching
Execution Time: 0 ms
Status: Completed successfully
Result: Nilai ditemukan pada indeks 2731
=====
Child Process 3:
PID: 2560
Task: Calculation
Execution Time: 0 ms
Status: Completed successfully
Result: Jumlah bilangan genap: 5087
=====
Semua child processes telah selesai!
asus@LAPTOP-BV04OHP1:~/tugas_os$

```

- Kompilasi:
 - Program dikompilasi menggunakan GCC compiler dengan perintah `gcc -o process_manager process_manager.c`
 - Flag `-o` menentukan nama output executable sebagai `"process_manager"`
 - compiler melakukan preprocessing, compilation, assembly, dan linking secara otomatis
 - Selama kompilasi, semua header files (`stdio.h`, `stdlib.h`, `unistd.h`, dll) di-include dan fungsi system call divalidasi
- Inisialisasi Eksekusi:
 - Program dijalankan dengan command `./process_manager` di Linux terminal

- OS loader mengalokasikan memory space dan load executable ke memory
- Parent process mulai eksekusi dari fungsi main() dengan PID original
- Array shared dengan 10.000 elemen dialokasikan dan diisi nilai random
- Pembuatan Processes:
- Parent process membuat multiple pipes menggunakan pipe() system call sebelum fork
- Tiga child processes dibuat secara sequential menggunakan fork() system call
- Setiap fork menghasilkan duplicate address space dengan copy-on-write semantics
- PID masing-masing process disimpan dalam array untuk tracking
- Eksekusi Paralel:
 - Setiap child process menjalankan task berbeda berdasarkan switch case index
 - Process 0 melakukan bubble sort dengan kompleksitas $O(n^2)$
 - Process 1 melakukan linear search dengan kompleksitas $O(n)$

- Process 2 menghitung bilangan genap dengan kompleksitas $O(n)$
- Waktu eksekusi diukur menggunakan `gettimeofday()` dengan presisi milidetik
- Koordinasi dan Komunikasi:
- Parent process memblokir eksekusi dengan `waitpid()` untuk setiap child
- Pipe communication mengirimkan structured data berformat
"PID|Task|Time|Result"
- Data parsing menggunakan `sscanf()` dengan delimiter pipe character
- Status termination dianalisis menggunakan macro `WIFEXITED` dan `WTERMSIG`
- Terminasi dan Cleanup:
 - Setiap child process mengirim hasil melalui pipe kemudian exit dengan status 0
 - Parent menutup pipe ends yang tidak digunakan untuk prevent resource leaks
 - Semua hasil dikumpulkan dan ditampilkan dalam formatted output
 - Program terminates setelah semua child processes complete execution

- Kode

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <time.h>

#define ARRAY_SIZE 10000
#define NUM_PROCESSES 3

// Fungsi untuk menghasilkan array acak
void generate_random_array(int arr[], int size) {
    for(int i = 0; i < size; i++) {
        arr[i] = rand() % 10000;
    }
}

// Fungsi sorting (Bubble Sort)
void bubble_sort(int arr[], int size) {
    for(int i = 0; i < size-1; i++) {
        for(int j = 0; j < size-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```



```
// Fungsi searching (Linear Search)
```

```
int linear_search(int arr[], int size, int target) {  
    for(int i = 0; i < size; i++) {  
        if(arr[i] == target) {  
            return i;  
        }  
    }  
    return -1;  
}
```

```
// Fungsi calculation (Menghitung jumlah bilangan genap)
```

```
int count_even_numbers(int arr[], int size) {  
    int count = 0;  
    for(int i = 0; i < size; i++) {  
        if(arr[i] % 2 == 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

```
// Fungsi untuk mendapatkan waktu saat ini dalam milidetik
```

```
long long current_time_millis() {  
    struct timeval tv;  
    gettimeofday(&tv, NULL);  
    return (long long)tv.tv_sec * 1000 + tv.tv_usec / 1000;  
}
```

```
int main() {
```

```
    pid_t pids[NUM_PROCESSES];
```

```
    int status;
```

```
    long long start_times[NUM_PROCESSES];
```

```
    long long end_times[NUM_PROCESSES];
```

```

int shared_array[ARRAY_SIZE];
generate_random_array(shared_array, ARRAY_SIZE);

printf("Parent Process PID: %d\n", getpid());
printf("Membuat %d child processes...\n\n", NUM_PROCESSES);

// Membuat pipe untuk komunikasi
int pipes[NUM_PROCESSES][2];
for(int i = 0; i < NUM_PROCESSES; i++) {
    if(pipe(pipes[i]) == -1) {
        perror("Pipe creation failed");
        exit(1);
    }
}

for(int i = 0; i < NUM_PROCESSES; i++) {
    start_times[i] = current_time_millis();
    pids[i] = fork();

    if(pids[i] == 0) {
        // Child process
        close(pipes[i][0]); // Tutup read end of pipe

        long long start = current_time_millis();
        int result;
        char task_name[20];

        switch(i) {
            case 0:
                sprintf(task_name, "Sorting");
                bubble_sort(shared_array, ARRAY_SIZE);
                result = shared_array[0]; // Ambil elemen pertama sebagai contoh
                printf("hasil\n");
                break;

```

```

        case 1:
            sprintf(task_name, "Searching");
            result = linear_search(shared_array, ARRAY_SIZE, 42); // Cari
nilai 42
            break;
        case 2:
            sprintf(task_name, "Calculation");
            result = count_even_numbers(shared_array, ARRAY_SIZE);
            break;
    }

    long long end = current_time_millis();
    long long duration = end - start;

    // Kirim hasil ke parent melalui pipe
    char buffer[256];
    snprintf(buffer, sizeof(buffer), "%d|%s|%lld|%d", getpid(), task_name,
duration, result);
    write(pipes[i][1], buffer, sizeof(buffer));
    close(pipes[i][1]);

    exit(0);
}
else if(pids[i] < 0) {
    perror("Fork failed");
    exit(1);
}
}

// Parent process menunggu dan mengumpulkan hasil
printf("Parent Process mengkoordinasi child processes...\n");

printf("=====\n");

```

```

for(int i = 0; i < NUM_PROCESSES; i++) {
    waitpid(pids[i], &status, 0);
    end_times[i] = current_time_millis();

    // Baca hasil dari pipe
    close(pipes[i][1]); // Tutup write end of pipe
    char buffer[256];
    read(pipes[i][0], buffer, sizeof(buffer));
    close(pipes[i][0]);

    // Parsing data dari pipe
    int child_pid;
    char task_name[20];
    long long duration;
    int result;

    sscanf(buffer, "%d|%^]|%lld|%d", &child_pid, task_name, &duration,
&result);

    printf("Child Process %d:\n", i+1);
    printf("  PID: %d\n", child_pid);
    printf("  Task: %s\n", task_name);
    printf("  Execution Time: %lld ms\n", duration);
    printf("  Status: ");

    if(WIFEXITED(status)) {
        printf("Completed successfully\n");
    } else if(WIFSIGNALED(status)) {
        printf("Terminated by signal %d\n", WTERMSIG(status));
    } else {
        printf("Unknown\n");
    }
}

```

```

printf(" Result: ");
switch(i) {
    case 0:
        printf("Array terurut (contoh elemen pertama: %d)\n", result);
        break;
    case 1:
        if(result != -1) {
            printf("Nilai ditemukan pada indeks %d\n", result);
        } else {
            printf("Nilai tidak ditemukan\n");
        }
        break;
    case 2:
        printf("Jumlah bilangan genap: %d\n", result);
        break;
}
printf("-----\n");
}

printf("Semua child processes telah selesai!\n");
return 0;
}

```

2.2 Chat Application dengan IPC

2.2.1 Membuat Chat Application dengan IPC

Program aplikasi chat ini mengimplementasikan arsitektur client-server terdistribusi yang sophisticated menggunakan shared memory sebagai media penyimpanan pesan terpusat dengan mekanisme semaphore untuk sinkronisasi antar proses. Sistem ini menggunakan struktur data `chat_room` yang terdefinisi dalam shared memory segment berisi array messages dua dimensi dengan kapasitas 10 pesan, `message_count` sebagai pointer jumlah pesan terkini, dan `read_count` untuk tracking pembacaan. Mekanisme sinkronisasi critical section diimplementasikan melalui binary semaphore menggunakan system calls `semget()`, `semctl()`, dan `semop()` dengan operasi atomic wait (-1) dan signal (+1) pada semaphore index 0. Setiap

proses pengguna mengakses shared memory melalui shmget() dengan key yang di-generate via ftok() berdasarkan file "chatfile", kemudian melakukan attachment memory space menggunakan shmat() untuk mendapatkan pointer ke struktur chat room.

Fungsi `send_message()` mengintegrasikan timestamp generation via `localtime()` dan format string menggunakan `snprintf()` sebelum menulis ke buffer shared memory dengan proteksi semaphore lock, sementara `show_history()` melakukan iterasi melalui seluruh message array untuk menampilkan chat history dengan mekanisme baca yang aman. Program ini mendemonstrasikan reliable inter-process communication dengan race condition prevention melalui proper semaphore management dan efficient memory sharing tanpa perlu network socket communication.

- Membuat file server.c,writer.c,reader.c

```
asus@LAPTOP-BV04OHPF:~$ cd Krisna  
asus@LAPTOP-BV04OHPF:~/Krisna$ nano server.c  
asus@LAPTOP-BV04OHPF:~/Krisna$ nano writer.c  
asus@LAPTOP-BV04OHPF:~/Krisna$ nano reader.c  
asus@LAPTOP-BV04OHPF:~/Krisna$ touch chatfile  
asus@LAPTOP-BV04OHPF:~/Krisna$ gcc -o server server.c  
asus@LAPTOP-BV04OHPF:~/Krisna$ ./server  
=== CHAT SERVER STARTING ===  
Using key: 1127274544  
Shared Memory ID: 0  
Semaphore ID: 0  
  
=== CHAT SERVER STARTED ===  
Server is running...  
Press Ctrl+C to shutdown server  
  
=== SERVER STATUS ===  
Messages: 0 | Active Users: 0  
=====
```

- Terminal 2

```

asus@LAPTOP-BV04OHP1: ~/Krisna
asus@LAPTOP-BV04OHP1:~/Krisna$ gcc -o writer writer.c
asus@LAPTOP-BV04OHP1:~/Krisna$ ./writer User1
Connecting to chat server...
Key: 1127274544
=== CHAT ROOM - Welcome User1 ===
Commands:
- Type messages to send
- 'history' to see all messages
- 'exit' to quit
User1> Hallo Krisna
Message sent: User1> Hallo Krisna
User1> Apa Hobimu
Message sent: User1> Apa Hobimu
User1> exit
Goodbye User1!
asus@LAPTOP-BV04OHP1:~/Krisna$

```

- Terminal 3

```

asus@LAPTOP-BV04OHP1: ~/Krisna
asus@LAPTOP-BV04OHP1:~/Krisna$ gcc -o writer writer.c
asus@LAPTOP-BV04OHP1:~/Krisna$ ./writer User2
Connecting to chat server...
Key: 1127274544
=== CHAT ROOM - Welcome User2 ===
Commands:
- Type messages to send
- 'history' to see all messages
- 'exit' to quit
User2> Halooo
Message sent: User2> Halooo
User2> Bermain Bulu Tangkis
Message sent: User2> Bermain Bulu Tangkis
User2> exit
Goodbye User2!
asus@LAPTOP-BV04OHP1:~/Krisna$

```

- Terminal 4

```

asus@LAPTOP-BV04OHP1: ~/Krisna
asus@LAPTOP-BV04OHP1:~/Krisna$ gcc -o reader reader.c
asus@LAPTOP-BV04OHP1:~/Krisna$ ./reader
Connecting to chat server...
Key: 1127274544
=== CHAT ROOM READER ===
Type 'refresh' to see new messages, 'exit' to quit
reader> refresh
=== NEW MESSAGES ===
User1> Hallo Krisna
User2> Halooo
User1> Apa Hobimu
User2> Bermain Bulu Tangkis
=====
reader> exit
Reader terminated.
asus@LAPTOP-BV04OHP1:~/Krisna$

```

2.2.2 Proses Kompilasi Dan Eksekusi

A. Persiapan Sistem

- Membuat file chatfile sebagai referensi kunci IPC
- Generate kunci unik menggunakan ftok() untuk shared memory dan semaphore

B. Kompilasi Program

- Kompilasi terpisah untuk server, writer, dan reader menggunakan GCC
- Menghasilkan tiga executable binary yang independen

C. Eksekusi Server

- Server dijalankan pertama kali untuk inisialisasi resources
- Membuat shared memory segment dan semaphore set
- Mengattach shared memory dan menginisialisasi struktur chat_room

D. Eksekusi Client

- Writer dan reader dijalankan dengan parameter username
- Client melakukan koneksi ke resources IPC yang sudah dibuat server
- Membuat attachment ke shared memory yang sama

E. Pertukaran Pesan

- Writer mengirim pesan dengan mengakuisisi write lock
- Reader membaca pesan menggunakan reader-writer pattern
- Pesan tersimpan dalam buffer shared memory yang terpusat

F. Sinkronisasi

- Semaphore menjamin mutual exclusion pada critical section
- Multiple readers dapat mengakses bersamaan, writer mendapatkan exclusive access
- Mencegah race condition dan menjaga konsistensi data

G. Terminasi

- Client exit dengan melepas memory attachment
- Server cleanup resources ketika dihentikan
- Semua processes termination secara graceful

H. Kode

- Kode untuk server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/sem.h>
```

```
#include <signal.h>
```

```
#define SHM_SIZE 1024
```

```
#define MAX_MESSAGES 10
```

```
#define MESSAGE_LENGTH 100
```

```
typedef struct {
```

```
    char messages[MAX_MESSAGES][MESSAGE_LENGTH];
```

```
    int message_count;
```

```
    int reader_count;
```

```
} chat_room;
```

```
int shmid, semid;
```

```
chat_room *chat;
```

```
void sem_op(int semid, int sem_num, int op) {  
  
    struct sembuf sb;  
  
    sb.sem_num = sem_num;  
  
    sb.sem_op = op;  
  
    sb.sem_flg = 0;  
  
    semop(semid, &sb, 1);  
  
}
```

```
void cleanup(int sig) {  
  
    printf("\nCleaning up...\n");
```

```
    if (chat != NULL) {  
  
        shmdt(chat);  
  
    }
```

```
    if (shmid != -1) {  
  
        shmctl(shmid, IPC_RMID, NULL);  
  
        printf("Shared memory removed\n");  
  
    }
```

```
    if (semid != -1) {  
  
        semctl(semid, 0, IPC_RMID);  
  
        printf("Semaphore removed\n");  
  
    }
```

```

printf("Server terminated.\n");

exit(0);
}

int main() {

    // Buat file untuk key

    system("touch chatfile 2>/dev/null");

    // Buat key

    key_t key = ftok("chatfile", 'C');

    if (key == -1) {

        printf("ftok failed, using default key 1234\n");

        key = 1234;

    }

    printf("=== CHAT SERVER STARTING ===\n");

    printf("Using key: %d\n", key);

    // Buat shared memory

    shmid = shmget(key, sizeof(chat_room), IPC_CREAT | 0666);

    if (shmid == -1) {

        perror("shmget failed");

        exit(1);
    }

```

```

}

printf("Shared Memory ID: %d\n", shmid);


// Attach shared memory

chat = shmat(shmid, NULL, 0);

if (chat == (void *)-1) {

    perror("shmat failed");

    exit(1);

}


// Initialize shared memory

memset(chat, 0, sizeof(chat_room));

chat->message_count = 0;

chat->reader_count = 0;


// Buat semaphore

semid = semget(key, 2, IPC_CREAT | 0666);

if (semid == -1) {

    perror("semget failed");

    exit(1);

}

printf("Semaphore ID: %d\n", semid);


// Initialize semaphores

```

```

semctl(semid, 0, SETVAL, 1); // mutex = 1

semctl(semid, 1, SETVAL, 1); // write_sem = 1


// Signal handling

signal(SIGINT, cleanup);

signal(SIGTERM, cleanup);


printf("\n=== CHAT SERVER STARTED ===\n");

printf("Server is running...\n");

printf("Press Ctrl+C to shutdown server\n\n");


// Server loop

while(1) {

    printf("=== SERVER STATUS ===\n");

    printf("Messages: %d | Active Users: %d\n", chat->message_count, chat-
>reader_count);

    printf("=====\n\n");

    sleep(3);

}

return 0;

}

```

- Kode writer.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/sem.h>
```

```
#define MAX_MESSAGES 10
```

```
#define MESSAGE_LENGTH 100
```

```
typedef struct {
```

```
    char messages[MAX_MESSAGES][MESSAGE_LENGTH];
```

```
    int message_count;
```

```
    int reader_count;
```

```
} chat_room;
```

```
int shmid, semid;
```

```
chat_room *chat;
```

```
void sem_op(int semid, int sem_num, int op) {
```

```
    struct sembuf sb;
```

```
    sb.sem_num = sem_num;
```

```

    sb.sem_op = op;

    sb.sem_flg = 0;

    semop(semid, &sb, 1);
}

int main(int argc, char *argv[]) {

    if (argc != 2) {

        printf("Usage: %s <username>\n", argv[0]);

        exit(1);

    }

    char *username = argv[1];

    // Create key (sama dengan server)

    key_t key = ftok("chatfile", 'C');

    if (key == -1) {

        key = 1234;

    }

    printf("Connecting to chat server...\n");

    printf("Key: %d\n", key);

    // Get shared memory

    shmid = shmget(key, sizeof(chat_room), 0666);

```

```

if (shmid == -1) {

    perror("Failed to access shared memory");

    printf("Please make sure the server is running first!\n");

    exit(1);

}


// Attach shared memory

chat = shmat(shmid, NULL, 0);

if (chat == (void *)-1) {

    perror("shmat failed");

    exit(1);

}


// Get semaphore

semid = semget(key, 2, 0666);

if (semid == -1) {

    perror("semget failed");

    exit(1);

}


printf("=== CHAT ROOM - Welcome %s ===\n", username);

printf("Commands:\n");

printf(" - Type messages to send\n");

printf(" - 'history' to see all messages\n");

```



```
printf(" - 'exit' to quit\n\n");
```

```
char input[80];
```

```
while(1) {
```

```
    printf("%s> ", username);
```

```
    fflush(stdout);
```

```
    if (fgets(input, sizeof(input), stdin) == NULL) {
```

```
        break;
```

```
    }
```

```
    // Remove newline
```

```
    input[strcspn(input, "\n")] = 0;
```

```
    if (strcmp(input, "exit") == 0) {
```

```
        break;
```

```
    }
```

```
    else if (strcmp(input, "history") == 0) {
```

```
        // Read messages
```

```
        sem_op(semid, 0, -1);
```

```
        chat->reader_count++;
```

```
        if (chat->reader_count == 1) {
```

```
            sem_op(semid, 1, -1);
```

```

    }

    sem_op(semid, 0, 1);

    printf("\n=== CHAT HISTORY ===\n");

    if (chat->message_count == 0) {

        printf("No messages yet.\n");

    } else {

        for (int i = 0; i < chat->message_count; i++) {

            printf("%s\n", chat->messages[i]);

        }

    }

    printf("=====\n\n");

    sem_op(semid, 0, -1);

    chat->reader_count--;

    if (chat->reader_count == 0) {

        sem_op(semid, 1, 1);

    }

    sem_op(semid, 0, 1);

}

else if (strlen(input) > 0) {

    // Send message

    sem_op(semid, 1, -1);

```

```

if (chat->message_count < MAX_MESSAGES) {

    // Construct message manually

    char message[MESSAGE_LENGTH];

    snprintf(message, MESSAGE_LENGTH, "%s> %s", username, input);

    message[MESSAGE_LENGTH-1] = '\0';


    strcpy(chat->messages[chat->message_count], message);

    chat->message_count++;

    printf("Message sent: %s\n", message);

} else {

    printf("Chat room is full! Cannot send message.\n");

}


sem_op(semid, 1, 1);

}

}


shmdt(chat);

printf("Goodbye %s!\n", username);

return 0;

}

```

- Kode reader.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/sem.h>
```

```
#define MAX_MESSAGES 10
```

```
#define MESSAGE_LENGTH 100
```

```
typedef struct {
```

```
    char messages[MAX_MESSAGES][MESSAGE_LENGTH];
```

```
    int message_count;
```

```
    int reader_count;
```

```
} chat_room;
```

```
int shmid, semid;
```

```
chat_room *chat;
```

```
void sem_op(int semid, int sem_num, int op) {
```

```
    struct sembuf sb;
```

```
    sb.sem_num = sem_num;
```

```
sb.sem_op = op;

sb.sem_flg = 0;

semop(semid, &sb, 1);

}
```

```
int main() {

    // Create key (sama dengan server)

    key_t key = ftok("chatfile", 'C');

    if (key == -1) {

        key = 1234;

    }


    printf("Connecting to chat server...\n");

    printf("Key: %d\n", key);


    // Get shared memory

    shmid = shmget(key, sizeof(chat_room), 0666);

    if (shmid == -1) {

        perror("Failed to access shared memory");

        printf("Please make sure the server is running first!\n");

        exit(1);

    }


    // Attach shared memory
```

```

chat = shmat(shmid, NULL, 0);

if (chat == (void *)-1) {

    perror("shmat failed");

    exit(1);

}


// Get semaphore

semid = semget(key, 2, 0666);

if (semid == -1) {

    perror("semget failed");

    exit(1);

}


printf("=== CHAT ROOM READER ===\n");

printf("Type 'refresh' to see new messages, 'exit' to quit\n\n");


int last_count = 0;

char input[10];


while(1) {

    printf("reader> ");

    fflush(stdout);


    if (fgets(input, sizeof(input), stdin) == NULL) {

```

```

        break;
    }

    input[strcspn(input, "\n")] = 0;

    if (strcmp(input, "exit") == 0) {
        break;
    }

    // Read messages

    sem_op(semid, 0, -1);

    chat->reader_count++;

    if (chat->reader_count == 1) {
        sem_op(semid, 1, -1);
    }

    sem_op(semid, 0, 1);

    if (chat->message_count > last_count) {
        printf("\n=== NEW MESSAGES ===\n");

        for (int i = last_count; i < chat->message_count; i++) {
            printf("%s\n", chat->messages[i]);
        }

        printf("=====\n\n");

        last_count = chat->message_count;
    }

```

```
    } else {  
        printf("No new messages.\n");  
    }  
  
    sem_op(semid, 0, -1);  
    chat->reader_count--;  
    if (chat->reader_count == 0) {  
        sem_op(semid, 1, 1);  
    }  
    sem_op(semid, 0, 1);  
}  
  
shmdt(chat);  
printf("Reader terminated.\n");  
return 0;  
}
```


BAB III

PENUTUP

Berdasarkan hasil implementasi dan analisis yang dilakukan, dapat disimpulkan bahwa praktikum sistem operasi ini berhasil mendemonstrasikan konsep fundamental manajemen proses dan komunikasi antar proses. Sistem manajemen proses yang diimplementasikan terbukti mampu mengkoordinasikan multiple child processes secara efektif melalui mekanisme pipe communication, dimana setiap proses child menjalankan tugas komputasi yang berbeda dengan akurasi waktu eksekusi yang terukur. Sementara itu, aplikasi chat yang dikembangkan menggunakan shared memory dengan sinkronisasi semaphore berhasil menyediakan platform komunikasi antar proses yang reliable dan aman dari race condition. Kedua implementasi ini tidak hanya mengilustrasikan konsep teoritis sistem operasi secara komprehensif, tetapi juga memberikan landasan yang kuat untuk pengembangan aplikasi distributed computing dan parallel processing yang lebih kompleks di masa depan.